

第 2 周 实验指导书

7.商品分类管理 API

(1) 添加商品分类

步骤 1：模型，在 models 目录下新建 CategoryModel.js 文件

```
module.exports=function(db,callback){  
  //分类模型  
  db.define("CategoryModel",{  
    cat_id:{type:'serial',key:true},  
    cat_name:String,  
    cat_pid:Number,  
    cat_level:Number,  
    cat_deleted:Boolean,  
    cat_icon:String,  
    cat_src:String  
  },{  
    table:"category"  
  });  
  return callback()  
}
```

步骤 2：数据访问层，在 Dao.js 中添加创建对象的方法

```
//创建对象数据  
module.exports.create=function(modelName,obj,cb){  
  var db=databaseModule.getDatabase();  
  var model=db.models[modelName];  
  model.create(obj,cb);//在 Model 中添加一条记录  
}
```

步骤 3：业务逻辑层，在 services 目录下创建 CategoryService.js 文件

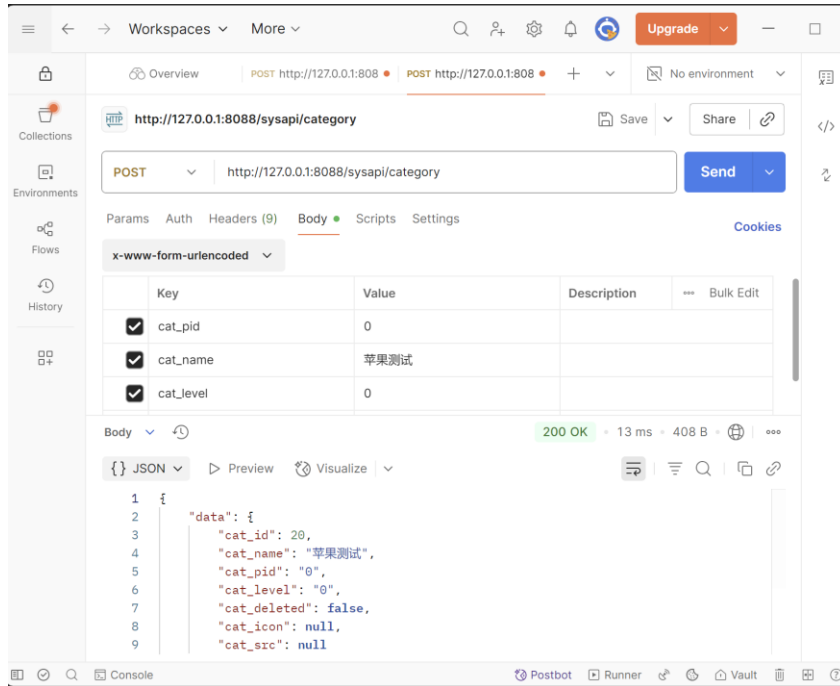
```
var path=require("path")  
var dao=require(path.join(process.cwd(),"dao/DAO"))  
module.exports.addCategory=function(cat,cb){  
  dao.create("CategoryModel",{  
    "cat_pid":cat.cat_pid,  
    "cat_name":cat.cat_name,  
    "cat_level":cat.cat_level  
  },function(err,newCat){  
    if(err) return cb("创建分类失败");  
    cb(null,newCat);  
  })  
}
```

步骤 4：路由层（对外暴露的接口），在 routes/sysapi/category.js 文件中添加创建分类的方法

```
var express=require('express')↵
var router=express.Router();↵
var path = require("path")↵
var categoryService=require(path.join(process.cwd(),"services/CategoryService"));↵
↵
router.get('/',(req,res)=>{↵
  res.sendResult('获取商品分类',200,'success')↵
})↵
↵
router.post("/",function(req,res,next){↵
  if(!req.body.cat_name){↵
    return res.sendResult(null,400,"必须提供分类名称")↵
  }↵
  next();↵
}, //业务逻辑↵
function(req,res){↵
  categoryService.addCategory({↵
    "cat_pid":req.body.cat_pid,↵
    "cat_name":req.body.cat_name,↵
    "cat_level":req.body.cat_level↵
  },function(err,result){↵
    if(err) return res.sendResult(null,400,err);↵
    res.sendResult(result,201,"创建成功")↵
  });↵
})↵
)↵
module.exports=router↵
```

步骤 5：测试

先登录，获得 token 后，再添加分类测试



category @fruit_shop (mysql) - 表

cat_id	cat_name	cat_pid	cat_level	cat_deleted	cat_icon	cat_src
5	水果	0	0	1	(Null)	(Null)
6	蔬菜	0	0	1	(Null)	(Null)
20	苹果测试	0	0	0	(Null)	(Null)
11	奇异果	0	0	0	(Null)	(Null)
12	葡萄莓果	0	0	0	(Null)	(Null)
13	柑橘橙柚	0	0	0	(Null)	(Null)
14	苹果蕉梨	0	0	0	(Null)	(Null)
15	榴莲	0	0	0	(Null)	(Null)
16	干果	0	0	0	(Null)	(Null)
17	礼盒果篮	0	0	0	(Null)	(Null)

(2) 获取全部分类列表

步骤 1: 数据访问层 Dao.js

```
//获取所有数据
module.exports.list=function(modelName,conditions){
  var db=databaseModule.getDatabase();
  var model=db.models[modelName];
  if(!model) return cb("模型不存在",null);
  if(conditions){
    if(conditions["columns"]){
      model=model.find(conditions["columns"]);
    }else{
      model=model.find();
    }
  }
  if(conditions["offset"]){
    model=model.offset(parseInt(conditions["offset"]));
  }
}
```

```

    }
    if(conditions["limit"]){
        model=model.limit(parseInt(conditions["limit"]));
    }
    if(conditions["only"]){
        model=model.only(parseInt(conditions["only"]));
    }
    if(conditions["omit"]){
        model=model.omit(conditions["omit"]);
    }
    if(conditions["order"]){
        model=model.order(conditions["order"]);
    }
}
}else{
    model=model.find();
}
model.run(function(err,models){
    if(err){
        return cb("查询失败",null);
    }
    cb(null,models);
})
}

```

步骤 2：业务逻辑层

数据返回的数组需要根据业务进行操作，Lodash 插件提供了很多便捷方法可以简化操作。安装 `npm i lodash`，在 `CategoryService.js` 中添加

```

...
var _=require('lodash')
...
//获取所有分类
module.exports.getAllCategories=function(type,conditions,cb){
    dao.list("CategoryModel",{cat_deleted:false},function(err,categories){
        var keyCategories=_keyBy(categories,'cat_id');
        //使用 Lodash 库的 _keyBy 方法将原来的 categories 数组转成一个键值对，
        //转换之后的每一个对象，其 key 值为 cat_id，值为一条记录
        if(!type) type=3;
        //转换为树形菜单，形如下图所示
        result=getTreeResult(keyCategories,categories,type);
        if(conditions){
            count=result.length;
            pagesize=parseInt(conditions.pagesize);
            pagenum=parseInt(conditions.pagenum)-1;
            result=_take(_drop(result,pagenum*pagesize),pagesize);
            var resultDta={};

```

```

        resultDta["total"]=count;
        resultDta["pagenum"]=pagenum;
        resultDta["pagesize"]=pagesize;
        resultDta["result"]=result;
        return cb(null,resultDta)
    }
    cb(null,result)
});
}

```

//获取树状结果

```

function getTreeResult(keyCategories,categories,type){
    var result=[];
    for(idx in categories){
        var cat=categories[idx];
        //判断是否被删除
        if(isDelete(keyCategories,cat)) continue;
        if(cat.cat_pid==0){
            result.push(cat);
        }else{
            if(cat.cat_level>=type) continue;
            var parantCat=keyCategories[cat.cat_pid];
            if(!parantCat) continue;
            if(!parantCat.children){
                parantCat["children"]=[];
            }
            parantCat.children.push(cat);
        }
    }
    return result;
}

```

//判断是否删除

```

function isDelete(keyCategories,cat){
    if(cat.cat_pid==0){
        return cat.cat_deleted;
    }else if(cat.cat_deleted){
        return true;
    }else{
        parentCat=keyCategories[cat.cat_pid];
        if(!parentCat) return true;
        return isDelete(keyCategories,parentCat);
    }
}

```

补充： 树形菜单

调用后可能返回:

```
javascript

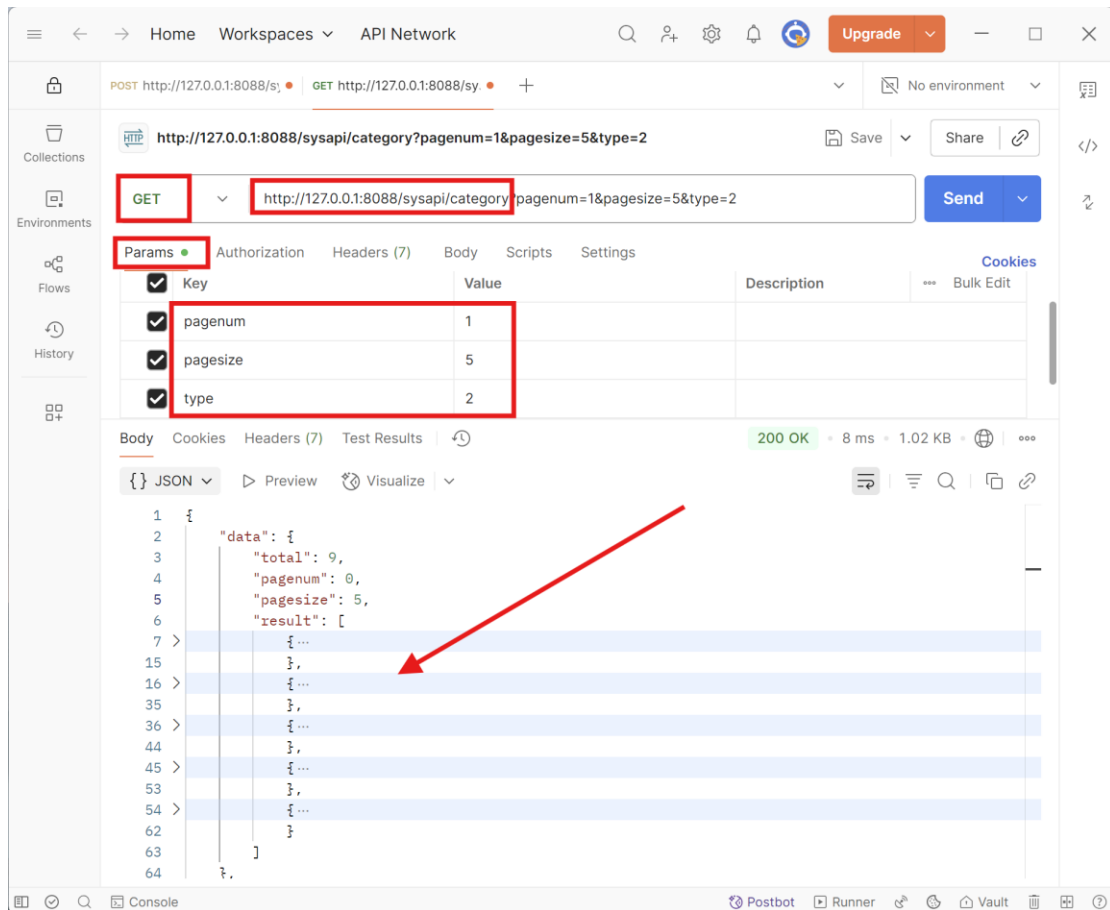
const result = [
  {
    id: 1,
    name: '电子产品',
    children: [
      { id: 2, name: '手机' },
      { id: 3, name: '电脑' }
    ]
  },
  {
    id: 4,
    name: '服装'
  }
];
```

步骤 3: 路由层, 在 routes/sysapi/category.js 文件中

特别注意: 将之前写的 `router.get('/',(req,res))` 注销或者删掉

```
//获取分类列表
router.get("/",function(req,res,next){
  next();//放行, 不校验
},function(req,res,next){
  var conditions=null;
  //之前判断用户传过来的值, post 方法通过 body 接收, get 方法通过 query 接收
  if(req.query.pagenum&&req.query.pagesize){
    conditions={
      "pagenum":req.query.pagenum,
      "pagesize":req.query.pagesize
    }
  }
  categoryService.getAllCategories(req.query.type,conditions,function(err,result){
    if(err) return res.sendResult(null,400,"获取分类列表失败");
    res.sendResult(result,200,"获取分类列表成功")
  })
});
```

步骤 4: 测试, 加上 token 后 【说明 type=2 代表只获取 1、2 级分类】



(3) 获取指定的分类

步骤 1: 数据访问层 Dao.js

```
//获取指定的分类，通过逐渐 ID 获取对象
module.exports.show=function(modelName,id,cb){
  var db=databaseModule.getDatabase();
  var model=db.models[modelName];
  model.get(id,function(err,obj){
    cb(err,obj);
  })
}
```

步骤 2: 业务逻辑层，在 CategoryService.js 中添加

```
//获得指定 ID 的分类对象
module.exports.getCategoryById=function(id,cb){
  dao.show("CategoryModel",id,function(err,category){
    if(err) return cb("获取分类对象失败");
    cb(null,category);
  })
}
```

步骤 3: 路由层，在 routes/sysapi/category.js 文件中

```
//获取指定 ID 的分类
router.get("/:id",function(req,res,next){
  if(!req.params.id){
```

```

        return res.sendResult(null,400,"分类 ID 不能为空");
    }
    if(isNaN(parseInt(req.params.id))) {
        return res.sendResult(null,400,"分类 ID 必须是数字");
    }
    next();
}, //正常业务逻辑
function(req,res,next){
    categoryService.getCategoryById(req.params.id,function(err,result){
        if(err) return res.sendResult(null,400,err);
        res.sendResult(result,200,"获取成功");
    })
}
)

```

步骤 4：测试【地址栏里直接写 5】

The screenshot shows a web browser's developer tools interface. The top bar indicates the current page is 'http://127.0.0.1:8088/sysapi/category/15'. The 'Headers' tab is selected, showing the following headers:

- Accept-Encoding: gzip, deflate, br
- Connection: keep-alive
- Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cC...

The 'Body' tab is also selected, showing the response in JSON format:

```

{
  "data": {
    "cat_id": 15,
    "cat_name": "榴莲",
    "cat_pid": 0,
    "cat_level": 0,
    "cat_deleted": false,
    "cat_icon": null,
    "cat_src": null
  },
  "meta": {
    "msg": "获取成功",
    "status": 200
  }
}

```

A red arrow points to the 'cat_id' field in the JSON response, which has the value 15.

(4) 修改指定的分类

步骤 1: 数据访问层 Dao.js

```
//修改指定的分类, 通过逐渐 ID 获取对象
module.exports.update=function(modelName,id,updateObj,cb){
  var db=databaseModule.getDatabase();
  var model=db.models[modelName];

  model.get(id,function(err,obj){
    if(err) return cb("更新失败",null);
    obj.save(updateObj,cb);
  })
}
```

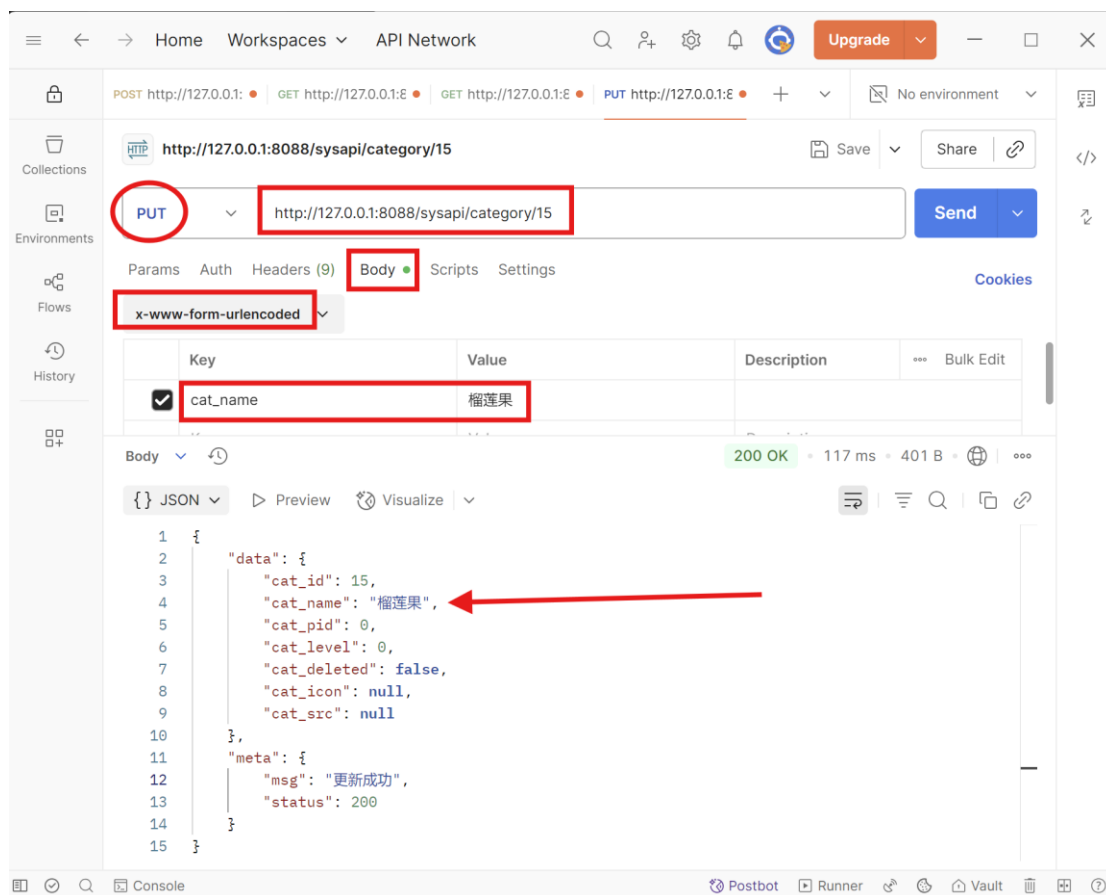
步骤 2: 业务逻辑层, 在 CategoryService.js 中添加

```
//更新指定 ID 的分类对象
module.exports.updateCategory=function(cat_id,newName,cb){
  dao.update("CategoryModel",cat_id,{"cat_name":newName},function(err,newCat){
    if(err) return cb("更新失败");
    cb(null,newCat);
  })
}
```

步骤 3: 路由层, 在 routes/sysapi/category.js 文件中

```
//更新指定 ID 的分类
router.put("/:id",function(req,res,next){
  if(!req.params.id){
    return res.sendResult(null,400,"分类 ID 不能为空");
  }
  if(isNaN(parseInt(req.params.id))){
    return res.sendResult(null,400,"分类 ID 必须是数字");
  }
  next();
}, //正常业务逻辑
function(req,res,next){
  categoryService.updateCategory(req.params.id,req.body.cat_name,function(err,result){
    if(err) return res.sendResult(null,400,err);
    res.sendResult(result,200,"更新成功");
  })
})
```

步骤 4: 测试,15 号“榴莲”更新成了“榴莲果”



(5) 删除指定的分类

步骤 1: 数据访问层 Dao.js 文件不需要改动, 分类软删除实际上是修改 `cat_deleted` 字段为 `true`, 因此直接调用之间的 `update` 方法即可。

步骤 2: 业务逻辑层, 在 CategoryService.js 中添加

```
//删除指定 ID 的分类对象
module.exports.deleteCategory=function(cat_id,cb){
  dao.update("CategoryModel",cat_id,{"cat_deleted":true},function(err,newCat){
    if(err) return cb("删除失败");
    cb("删除成功");
  })
}
```

步骤 3: 路由层, 在 routes/sysapi/category.js 文件中

```
//删除指定的分类
router.delete("/:id",function(req,res,next){
  if(!req.params.id){
    return res.sendResult(null,400,"分类 ID 不能为空");
  }
  if(isNaN(req.params.id)){
    return res.sendResult(null,400,"分类 ID 必须是数字");
  }
  next();
},function(req,res,next){
```

```
  categoryService.deleteCategory (req.params.id,function(err,result){
    if(err) return res.sendResult(null,400,err);
    res.sendResult(result,200,"删除指定分类成功");
  })
})
```

步骤 4: 测试

The screenshot shows a REST client interface with the following details:

- URL: `http://127.0.0.1:8088/sysapi/category/15`
- Method: `DELETE`
- Response: `200 OK` (17 ms, 291 B)
- Body (JSON):

```
{
  "data": null,
  "meta": {
    "msg": "删除成功",
    "status": 200
  }
}
```

category @fruit_shop (mysql) - 表						
文件 编辑 查看 窗口 帮助						
导入向导 导出向导 筛选向导 网格查看 表单查看 备注 十六进制 图像 A-Z 升序排序						
cat_id	cat_name	cat_pid	cat_level	cat_deleted	cat_icon	cat_src
5	水果	0	0	0	1 (Null)	(Null)
6	蔬菜	0	0	0	1 (Null)	(Null)
20	苹果测试	0	0	0	0 (Null)	(Null)
21	测试1	0	0	0	0 (Null)	(Null)
22	测试1-1	21	0	0	0 (Null)	(Null)
11	奇异果	0	0	0	0 (Null)	(Null)
12	葡萄莓果	0	0	0	0 (Null)	(Null)
13	柑橘橙柚	0	0	0	0 (Null)	(Null)
14	苹果蕉梨	0	0	0	0 (Null)	(Null)
15	榴莲果	0	0	1	1 (Null)	(Null)
16	干果	0	0	0	0 (Null)	(Null)
17	礼盒果篮	0	0	0	0 (Null)	(Null)
23	删除测试	0	0	0	0 (Null)	(Null)

SELECT * FROM `category` LIMIT 第 13 条记录 (共 13 条) 于 1 页